

ERWYNA. Task 1 serial, plus the benchmarking and graphs

1. Walk me through IsPrime. Weak: "It checks if a number is prime." Strong: Handle k below 2, then 2 itself, then reject all even numbers. Compute limit as the integer square root of k. Loop d from 3 to limit stepping by 2, and if any d divides k return not prime. Reaching the end means prime.
2. Why stop at the square root and not at k? Weak: "Because it is faster." Strong: If k is composite it factorises as $k = a \times b$. Both cannot exceed the square root, because then a times b would exceed k. So at least one factor is at or below the square root. Finding none there proves k is prime. It cuts the work from order k to order square root of k per candidate.
3. Why step by 2 in the divisor loop? Strong: We already rejected every even k before the loop. An even divisor cannot divide an odd number, so testing them is wasted work. Stepping by 2 halves the divisions.
4. Why a flag array instead of a list of primes? Strong: Reading the array front to back gives ascending order with no sort step, which the spec requires. It also means each thread writes only to its own indices in Tasks 2 and 3, so there is no race and no mutex. The cost is n bytes even for non primes, so 30 MB at n equals 30 million. Follow up they may ask: how would you cut the memory? Use one bit per number instead of one byte, and pay for it with masking work on every access.
5. Why does the timer stop before the file write? Strong: Writing 664,579 lines is serial disk work that no number of threads can speed up. Including it would drag every speedup number down and understate the parallel gain. It is Amdahl's Law made visible, so we report it separately.
6. Why CLOCK_MONOTONIC rather than CLOCK_REALTIME? Strong: Monotonic cannot jump if the system clock is adjusted mid run, so an elapsed time measured with it is always a real elapsed duration.
7. How do you know your three programs are correct? Strong: All three write their primes to a file, we diff the three files and they match exactly, and the count is 664,579 which is the known number of primes below ten million. The benchmark script runs that check before it times anything, so a broken build cannot produce timings.
8. Why best of three runs rather than the average? Strong: Interference is one sided. Background processes can only make a run slower, never faster. An average pulls in whatever else the laptop was doing, while the fastest run is the closest we get to the machine to itself.
9. Why 30 values of n, and why threads up to 28? Strong: The spec asks for at least 30 values of n for statistically convincing evidence, and asks that threads go from 1 to at least the core count. We have 14 cores so we swept to 28, twice the core count, to show what happens past it.

10. Point at Graph 4 and explain the shape. Strong: Near one to one to 8 threads, 2.00x at 2, 3.93x at 4, 7.79x at 8. Then it bends and plateaus just under 10x at the core count of 14. Past 14 the extra threads take turns on the same cores, so there is no more hardware parallelism to exploit and the line stays flat.

11. Your speedup is 9.17x on 14 cores. Why not 14x? Strong: Two reasons. Amdahl's Law, the serial fraction, memory allocation, thread creation and the join, caps speedup regardless of thread count. And parallel overhead plus load imbalance, since scheduling costs time that does no searching and threads never finish together. On this chip 10 cores are performance and 4 are efficiency, so a true 14x was never available.

TAABISH. Task 2 POSIX Threads and Task 3 OpenMP

1. Walk me through how a thread gets created and joined. Strong: `pthread_create` takes the thread id, attributes which we leave null, the function to run, and one void pointer argument. We create `g_numThreads` of them in a loop, then a second loop calls `pthread_join` on each so main waits for all of them before touching the results. That is the fork and join pattern.

2. Why is there a `threadNum` array instead of passing the address of `i`? Strong: Passing `&i` gives every thread a pointer to the same variable while the loop is still incrementing it. Threads then read whatever value `i` happens to hold when they start, so some get the wrong rank and duplicate each other's chunks. That is a genuine data race. Giving each thread its own slot means the value cannot change underneath it.

3. Explain your partitioning scheme. Strong: Cyclic chunk distribution. Each thread starts at chunk index equal to its rank and advances by the thread count, taking 1000 numbers per chunk, wrapping until the range runs out. So thread 0 takes chunk 0, thread 1 takes chunk 1 and so on, then it repeats.

4. Why is it better than one block per thread? Strong: Cost per candidate grows with the square root, so high numbers are more expensive. With one block each, the thread holding the top of the range is still working long after the others finish and everyone waits at the join. That is load imbalance. Cyclic gives every thread a mix of cheap and expensive candidates so they finish together. Measured, 8.70x became 9.75x.

5. Why chunk size 1000? Strong: Trade off both ways. Too small and loop bookkeeping dominates and threads keep touching the same cache lines. Too big and it degenerates into the block version with one thread holding a long run of expensive numbers. At 1000 with 14 threads and `n` equal to 30 million there are still 30,000 chunks circulating, so the work evens out. Anything from a few hundred to a few thousand behaved the same.

6. Where is your critical section, and why is there no mutex? Strong: There is none in the parallel region, deliberately. Each thread writes only to indices it alone owns, so the writes are disjoint and no race exists. A mutex would serialise the hot loop and destroy the speedup for no correctness gain. The only shared state, `n` and the thread count, is read only after setup.

7. Is there really no sharing at all between threads? Strong: At byte level no. But one cache line covers 64 of these bytes, so threads working on adjacent chunks can share a line at the borders. That is false sharing. It can cost a little performance but it is not a correctness problem, and chunks of 1000 make the borders rare enough that it does not show in the timings.

8. In Task 3, what is private and what is shared? Strong: `k` is the loop variable of the parallel for so OpenMP makes it private automatically. `n` and `pFlags` stay shared, which is what we want, since `n` is only read and the threads write to different slots of `pFlags`.

9. What if you needed a running total inside the parallel loop? Strong: A reduction clause, reduction plus on the counter. Not a critical section, because that would serialise every iteration. Reduction gives each thread a private partial total and combines them at the end.

10. Explain the three OpenMP schedules and why you chose dynamic. Strong: Static divides the iterations up before the loop starts. Dynamic lets each thread take the next free chunk as it finishes one. Guided starts with large chunks and shrinks them. Our workload is skewed toward expensive high candidates, so dynamic fits, and unlike our hand written rotation it adapts at run time. Measured at n equal to 30 million, static 9.55x, dynamic 10.44x, guided 10.42x.

11. Would you recommend OpenMP over POSIX Threads? Strong: For this workload yes. One pragma replaced about forty lines of thread management and came out slightly faster, 9.37x against 8.09x on average. It also supports incremental parallelism and compiling without the flag gives the serial version back. I would choose POSIX Threads where the parallelism is not loop shaped, a producer consumer pipeline for instance, or where I need control over thread lifetime, affinity or per thread state.

12. What happens if you forget the fopenmp flag? Strong: The compiler ignores the pragma, the program still compiles and runs, but on one thread. Speedup reads exactly 1.00 with no warning, so it is easy to miss.